1. What is variable?

A variable has a name, and you can store something in it.

A variable is a named storage location in memory that holds a value which can be changed or updated during execution.

3. Primitive vs Non-Primitive Data Types

Primitive:

Stored directly in memory

Immutable

Types:

string, number, boolean, undefined, null, symbol, bigint

Non-Primitive:

Stored as reference (memory address)

Types:

object, array, function, date, regex

👉 Copy karte time reference copy hota hai, actual value nahi

4. What is null & undefined?

null → intentional empty value

undefined → variable declared but value not assigned

5. typeof in JavaScript

typeof ek operator hai jo variable ka data type batata hai (string return karta hai).

6. What is this keyword?

Current executing object ko refer karta hai

Browser → window object

Node.js → global object

9. Difference: Normal vs Arrow Function

Normal Function

Own this

Constructor ban sakta hai

Arrow Function

Lexical this

Constructor nahi hota

Best for callbacks

10. What is anonymous function?

Function without name

Example:

```
const greet = function() {
  console.log("Hello");
}
```

7. What is super constructor?

super() use hota hai child class me parent constructor ko call karne ke liye

Uses:

Parent constructor call

Parent methods access

8. What is closure?

Closure = function + uska lexical environment

👉 Inner function outer function ke variables ko access aur remember karta hai even after outer function execute ho gaya ho.

11. What is Debouncing?

Debouncing ek technique hai jo function ko repeatedly call hone se rokta hai

👉 Delay karta hai execution (scroll, search input etc.)

12. What is Currying?

Function ko multiple arguments se convert karke chain of functions banana

Example:

```
const multiply = a => b => c => a * b * c;
multiply(2)(3)(4);
```

13. What is Hoisting?

JavaScript declarations ko top pe move karta hai execution se pehle

var → hoisted (default undefined)

let/const → TDZ me rehte hai

14. Temporal Dead Zone (TDZ)

Time between variable hoisting and initialization

👉 let/const access nahi kar sakte initialization se pehle

15. Getter & Setter

Getter → value get karta hai

Setter → value update/set karta hai

16. Event Loop in JavaScript

JavaScript asynchronous kaam handle karta hai

Main parts:

Call Stack

Callback Queue

👉 Non-blocking execution allow karta hai

17. Microtask vs Macrotask

Microtask:

Promise.then(), catch(), finally()

queueMicrotask()

Macrotask:

setTimeout, setInterval

setImmediate

requestAnimationFrame

I/O events

👉 Microtasks pehle execute hote hai

1. Event Loop Execution Order

Call Stack → Microtask Queue → Macrotask Queue

👉 Flow:

Call stack empty

All microtasks run

One macrotask run

Repeat

2. Rest vs Spread Operator

Rest (...)

Multiple elements ko array me collect karta hai

Used in function parameters

Spread (...)

Array/object ko expand karta hai

Used in arrays, objects, function calls

3. Destructuring in JavaScript

Objects/arrays se values extract karke variables me store karna

👉 Code short aur readable ho jata hai

4. Slice vs Splice

slice(start, end)

New array return karta hai

Original array change nahi hota

const arr = [1,2,3,4,5];

arr.slice(1,4); // [2,3,4]

splice(start, deleteCount, items)

Original array modify karta hai

arr.splice(2,1,"a");

5. Call vs Apply

call()

Arguments alag-alag pass hote hai

apply()

Arguments array me pass hote hai

6. setImmediate vs setTimeout

setTimeout(fn, delay)

Delay ke baad execute

setImmediate(fn)

Current event loop ke baad execute

7. find vs filter

find()

First matching element return

filter()

All matching elements ka new array return

8. shift vs unshift

shift()

First element remove karta hai

unshift()

Start me element add karta hai

9. push vs concat

push()

Original array modify karta hai

concat()

New array return karta hai (original safe)

10. forEach vs map

forEach()

Loop only (no return)

map()

New array return karta hai

11. reduce()

Array ke elements ko combine karke single value deta hai

```
const total = [10,20,30].reduce((acc,curr)=>acc+curr,0);
```

12. stopPropagation()

Event bubbling/capturing ko stop karta hai

13. import vs require

require()

CommonJS

Synchronous

Anywhere use kar sakte ho

import

ES Modules

Asynchronous

Top level pe use hota hai

14. Deep Copy vs Shallow Copy

Shallow Copy

Top-level copy

Nested objects same reference

Deep Copy

Full copy (no reference shared)

JSON.parse(JSON.stringify(obj))

15. Authentication vs Authorization

Authentication

"Who are you?" (identity verify)

Authorization

"What can you do?" (permissions)

16. Stateless vs Stateful

Stateless

No internal state

Only props use karta hai

Stateful

State manage karta hai

Data change hota hai

17. ES6 Features

let, const

Arrow functions

Template literals

Default parameters

Destructuring

Classes

Promises

Map, Set, WeakMap, WeakSet

Symbols

18. async / await

async

Function Promise return karta hai

await

Promise resolve hone tak wait karta hai

20. Session in JavaScript

Session = temporary interaction between client & server

👉 Data store hota hai:

Session storage me

Limited time ke liye

1. Polymorphism in JavaScript

Polymorphism ka matlab same function/method different behavior dikhata hai depending on object

👉 JavaScript me:

Method overriding supported

Method overloading directly supported nahi hai

2. Session in JavaScript

Session = client (browser) aur server ke beech temporary interaction

👉 JavaScript me:

sessionStorage use hota hai

Data ek browser session ke liye store hota hai

Example:

```
sessionStorage.setItem("username", "Alice");
let user = sessionStorage.getItem("username");
sessionStorage.removeItem("username");
sessionStorage.clear();
```

3. Cookie in JavaScript

Cookie = small data jo browser me store hota hai aur server ko request ke sath bheja jata hai

Example:

```
document.cookie = "username=John; expires=Fri, 31 Dec 2025 23:59:59 GMT";
console.log(document.cookie);
```

4. Promise vs Observable

Promise

Single value return

Immediately execute hota hai

Cancel nahi kar sakte

.then(), .catch(), .finally()

Observable

Multiple values emit kar sakta hai

Lazy (subscribe hone pe run hota hai)

Cancel possible

RxJS operators (map, filter, switchMap)

5. Types of Promises

Promise.all()

Promise.allSettled()

Promise.race()

Promise.any()

6. Multiple API Calls Handle

Promises:

Promise.all([api1, api2])

Observables:

forkJoin

switchMap

7. Dependent API Calls

Promises:

async/await

chained promises

Observables:

switchMap

concatMap

mergeMap

8. Single Response from Multiple APIs

Promises:

Promise.all()

Promise.allSettled()

Observables:

forkJoin

switchMap

9. Memory Leak

Memory leak tab hota hai jab memory use ho rahi ho but release nahi ho rahi

👉 Result:

App slow ho jata hai

Crash ho sakta hai

10. Prototype in JavaScript

Prototype = mechanism jisse objects inheritance karte hai

👉 Har object ka internal [[Prototype]] hota hai

11. Webpack vs Babel

Webpack

Module bundler

Files (JS, CSS, images) bundle karta hai

Babel

JavaScript compiler

ES6+ ko old browser compatible banata hai

12. TypeScript

TypeScript = JavaScript + Types

👉 Features:

Static typing

Interfaces

Classes

Type checking

13. Interface vs Type

Interface

Object structure define karta hai

Extend kar sakte hai

Type

Flexible

Unions, intersections support karta hai

14. Getter & Setter (Advanced)

Property access control karta hai

Encapsulation improve karta hai

🔥 OOPS Concepts

15. Abstraction

Sirf important details dikhana, unnecessary hide karna

16. Encapsulation

Data + functions ko ek unit me bind karna

1. What is HTML?

👉 HTML (HyperText Markup Language) is the standard language used to create the structure of web pages.

It defines elements like headings, paragraphs, images, links, and forms using tags.

2. Difference between HTML & CSS?

👉 HTML is used to create structure (content), while CSS is used to style that content (colors, layout, fonts, spacing).

3. What is JavaScript?

👉 JavaScript is a scripting language used to make web pages dynamic and interactive.

It can handle events, manipulate DOM, call APIs, and perform logic.

4. What is DOM?

👉 DOM (Document Object Model) is a tree-like structure of HTML elements.

JavaScript uses DOM to access and modify elements dynamically.

5. Difference between var, let, const?

var → function scoped, can be redeclared

let → block scoped, cannot redeclare

const → block scoped, cannot reassign

👉 Preferred: use let and const

6. What is closure?

👉 Closure is when a function remembers variables from its outer scope even after the outer function has finished execution.

Used in data hiding and callbacks.

7. What is event bubbling?

👉 When an event occurs on an element, it first runs on that element, then bubbles up to parent elements.

8. What is Promise?

👉 A Promise represents an asynchronous operation that may complete in future.

States: pending → resolved → rejected

9. What is async/await?

👉 async/await is a cleaner way to handle promises.

It makes asynchronous code look like synchronous code.

10. Difference between == and ===?

== → compares values (type conversion allowed)

=== → compares value + type (strict comparison)

React JS Interview Q&A

1. Key Features of React

Component-based architecture

Virtual DOM

JSX

One-way data flow

2. What is JSX?

JSX = JavaScript XML

👉 JavaScript ke andar HTML-like code likhne ka syntax

3. What are Props?

Props = parent → child data pass karne ka tarika

👉 Read-only (immutable) hote hai

11. What is React?

👉 React is a JavaScript library used to build user interfaces, especially single-page applications (SPA).

It uses component-based architecture.

12. What is component?

👉 A component is a reusable piece of UI.

Types:

Functional component (modern)

Class component (older)

4. What is State?

State = component ka internal data

👉 Dynamic hota hai, change hone par UI re-render hota hai

5. State vs Props

State

Mutable

Component ke andar manage hota hai

UI change karta hai

Props

Immutable

Parent control karta hai

Data pass karta hai

13. What is state?

👉 State is data that belongs to a component and can change over time.

When state updates → component re-renders.

14. What are props?

👉 Props are inputs passed from parent component to child component.

They are read-only.

15. Difference between state & props?

state → managed inside component

props → passed from parent

7. Functional vs Class Components

Functional

Simple JS functions

Hooks use karte hai

Class

ES6 classes

lifecycle methods use karte hai

8. React.Component vs React.PureComponent

Component

Always re-render

PureComponent

Shallow comparison karta hai

Unnecessary re-render avoid karta hai

18. What is Virtual DOM?

👉 Virtual DOM is a lightweight copy of the real DOM.

React updates only changed parts → improves performance.

19. What is React Router?

👉 Used for navigation between pages without reloading.

20. What is key in React?

👉 A unique identifier used when rendering lists to help React track elements.

🖥️ SECTION 3: NODE.js + EXPRESS

21. What is Node.js?

👉 Node.js is a runtime environment that allows JavaScript to run on the server.

It is built on Chrome's V8 engine.

22. What is Express.js?

👉 Express is a lightweight framework for Node.js used to build APIs and web applications.

23. What is middleware?

👉 Middleware is a function that runs between request and response cycle.

Example: authentication, logging

24. What is REST API?

👉 REST API is a way of communication using HTTP methods:

GET → read

POST → create

PUT → update

DELETE → remove

25. Difference between GET & POST?

GET → data in URL, used to fetch

POST → data in body, used to send/store

26. What is JWT?

👉 JSON Web Token is used for authentication.

It stores user info securely and is sent with every request.

27. Authentication vs Authorization?

Authentication → verify identity

Authorization → check permissions

28. What is body-parser?

👉 Middleware to parse incoming request body (JSON).

29. What is CORS?

👉 Cross-Origin Resource Sharing allows frontend & backend to communicate from different domains.

30. What are HTTP status codes?

200 → success

404 → not found

500 → server error

11. Reconciliation in React

Reconciliation = DOM update process

👉 React diffing algorithm use karta hai fast updates ke liye

12. React Fiber

React Fiber = new rendering engine

👉 Features:

Incremental rendering

Better performance

Priority-based updates

13. createElement vs cloneElement

createElement

New element create

cloneElement

Existing element copy + modify

1. React.memo vs useMemo

React.memo

Component ko re-render hone se bachata hai

Props change na ho to render skip

useMemo

Value ko cache karta hai

Heavy calculation optimize karta hai

6. Higher Order Component (HOC)

HOC = function jo component ko input me lekar new enhanced component return karta hai

👉 Reusability ke liye use hota hai

8. React Fragments

Multiple elements return karne ke liye bina extra div ke

```
<>
  <h1>Hello</h1>
  <p>World</p>
</>
```

9. Batching in React

Multiple state updates ko combine karke ek hi render me execute karta hai

👉 Performance improve hoti hai

10. How React Works (Flow)

JSX → JS (createElement)

Virtual DOM create

Diffing

Real DOM update

11. Synthetic Events

React ka wrapper for browser events

👉 Cross-browser compatible

12. Flux in React

Flux = architecture pattern

👉 One-way data flow

👉 Predictable state management

13. State Lifting

State ko parent component me move karna

14. Redux Form

Redux Form = form state manage karne ki library

👉 Handle karta hai:

Values

Validation

Submission

15. Next.js

Next.js = React framework

👉 Features:

Server-side rendering (SSR)

Routing

SEO optimization

Full-stack support

⚡ RxJS Interview Q&A

16. What is RxJS?

RxJS = Reactive Extensions for JavaScript

👉 Asynchronous data streams handle karta hai

17. Subject vs BehaviorSubject

Subject

Current value store nahi karta

BehaviorSubject

Latest value store karta hai

New subscribers ko last value milti hai

18. Cold vs Hot Observables

Cold

Har subscriber ke liye new execution

Hot

Shared execution

Same data sabko milta hai

19. RxJS Operators

of

from

map

mergeMap

switchMap

filter

take

merge

concat

forkJoin

20. takeUntil Operator

Stream ko stop (unsubscribe) karta hai

👉 Angular me memory leak avoid karne ke liye use hota hai

21. switchMap vs mergeMap

mergeMap

Parallel execution

No cancellation

switchMap

Previous request cancel karta hai

Latest request hi run hota hai

31. What is MongoDB?

👉 MongoDB is a NoSQL database that stores data in JSON-like format (BSON).

It is flexible and scalable.

32. What is collection?

👉 A collection is a group of documents (similar to table).

33. What is document?

👉 A document is a single record stored as JSON object.

34. SQL vs NoSQL?

SQL → structured, tables

NoSQL → flexible, document-based

35. What is Mongoose?

👉 Mongoose is an ODM library to interact with MongoDB using schemas.

36. What is schema?

👉 Schema defines structure of a document (fields, types, validations).

37. What is CRUD?

👉 Create, Read, Update, Delete operations.

38. What is indexing?

👉 Improves query performance by creating fast lookup.

41. How frontend connects to backend?

👉 Using API calls via Axios or fetch.

43. What is API?

👉 API is a bridge that allows two systems to communicate.

44. What is .env file?

👉 Stores secret data like API keys, DB URL.

45. What is deployment?

👉 Hosting app on server (Vercel, Netlify, Render).

1. What is controlled vs uncontrolled component?

👉 Controlled Component

Form data handled by React state

Example: input value stored in useState

👉 Uncontrolled Component

Form data handled by DOM (using ref)

✅ Interview Tip:

Say → “Controlled is preferred because it gives better control and validation.”

2. What is lifting state up?

👉 When multiple components need same data, move state to their common parent.

👉 Example:

Cart count shared between Navbar & Product page.

3. What is prop drilling?

👉 Passing data through multiple layers of components unnecessarily.

👉 Solution:

Context API

Redux

4. What is Context API?

👉 Used to share data globally without passing props manually.

👉 Example:

User login data

Theme

5. What is useRef?

👉 Used to:

Access DOM elements

Store mutable values without re-render

6. What is useMemo?

👉 Optimizes performance by memoizing values.

👉 Prevents unnecessary recalculations.

7. What is useCallback?

👉 Memoizes functions to prevent re-creation on every render.

8. What is lazy loading in React?

👉 Loads components only when needed.

👉 Improves performance.

9. What is React lifecycle?

👉 Phases:

Mounting

Updating

Unmounting

(Hooks handle lifecycle in functional components)

10. What is reconciliation?

👉 Process React uses to update DOM efficiently using Virtual DOM diffing.

11. What is event loop in Node.js?

👉 Event loop handles asynchronous operations in Node.js.

👉 It allows non-blocking execution.

12. What is non-blocking I/O?

👉 Operations do not stop execution of other code.

👉 Example:

Reading file without blocking server.

13. What is clustering in Node.js?

👉 Running multiple Node instances to utilize CPU cores.

14. What is rate limiting?

👉 Prevents too many requests from a user.

👉 Used for:

Security

Prevent DDoS

15. What is helmet.js?

👉 Middleware to secure HTTP headers.

16. What is express-session?

👉 Used to manage user sessions on server.

17. What is bcrypt?

👉 Library used to hash passwords securely.

18. What is MVC architecture?

👉 Model → data

👉 View → UI

👉 Controller → logic

19. What is error handling middleware?

👉 Middleware used to handle errors globally.

20. What is API throttling?

👉 Limits number of API requests in a time frame.

🍃 MONGODB (ADVANCED)

21. What is aggregation?

👉 Used for complex queries like grouping, filtering, calculations.

22. What is $lookup?

👉 Used to join collections (like SQL JOIN).

23. What is sharding?

👉 Splitting database across multiple servers for scalability.

24. What is replication?

👉 Copying data to multiple servers for backup & availability.

25. What is ObjectId?

👉 Unique ID generated for each document.

26. What is lean() in Mongoose?

👉 Returns plain JS object → improves performance.

27. What is population?

👉 Used to replace reference with actual document.

28. What is validation in Mongoose?

👉 Ensures correct data format before saving.

29. What is indexing types?

👉 Single field, compound, text index.

30. What is TTL index?

👉 Automatically deletes data after certain time.

🔗 FULL STACK (REAL WORLD)

31. How do you secure your application?

👉 Use:

JWT authentication

Password hashing

HTTPS

CORS

Rate limiting

33. What is caching?

👉 Storing data temporarily to reduce API calls.

👉 Tools:

Redis

Browser cache

34. What is load balancing?

👉 Distributes traffic across servers.

35. What is microservices?

👉 Application divided into smaller independent services.

36. What is monolithic architecture?

👉 Single large application.

37. What is API versioning?

👉 Managing different API versions (v1, v2).

38. What is CI/CD?

👉 Continuous Integration & Deployment (automated build & deploy).

39. What is Docker?

👉 Tool to containerize applications.

40. What is environment-based config?

👉 Different configs for dev, test, production.

42. How to handle large traffic?

👉 Use:

Load balancing

Caching

CDN

43. How to handle file upload?

👉 Use:

Multer (Node.js)

Store in cloud (AWS, Cloudinary)

44. How to improve API speed?

👉 Use:

Indexing

Caching

Reduce DB queries

45. How to debug issue in production?

👉 Use:

Logs

Monitoring tools

Error tracking

16. What is SaaS?

SaaS = Software as a Service

👉 Software internet pe available hota hai

👉 Install karne ki need nahi

17. What is SCSS?

SCSS = CSS preprocessor

👉 Features:

Variables

Nesting

Mixins

👉 CSS ko powerful banata hai

18. What is Docker?

Docker = containerization platform

👉 App + dependencies ko ek container me package karta hai

1. What is RxJS?

RxJS (Reactive Extensions for JavaScript) ek library hai jo asynchronous data streams handle karti hai

👉 Use cases:

API calls

User events

Real-time data

2. Subject vs BehaviorSubject

Subject

Initial value nahi hoti

Sirf new values milti hai

BehaviorSubject

Initial value hoti hai

Latest value new subscriber ko milti hai

3. Cold vs Hot Observables

Cold Observable

Har subscriber ke liye new execution

Hot Observable

Shared execution

Same data sabko milta hai

4. RxJS Operators List

of, from

map, filter

mergeMap, switchMap

take

merge, concat

forkJoin

5. takeUntil

Stream ko automatically stop (unsubscribe) karta hai

👉 Angular me memory leak avoid karne ke liye

6. switchMap vs mergeMap

mergeMap

Parallel execution

Old request cancel nahi hoti

switchMap

Previous request cancel karta hai

Latest request hi run hoti hai

7. concat vs merge

concat

Sequential execution

Ek ke baad ek

merge

Parallel execution

Order maintain nahi hota

8. forkJoin

Multiple observables complete hone ke baad ek result deta hai

👉 Final combined output

9. Error Handling in RxJS

catchError

retry

finalize

10. shareReplay

Cold observable → Hot banata hai

Last value cache karta hai

Duplicate API calls avoid karta hai

🌐 HTML & CSS Interview Q&A

11. Semantic Elements

Semantic HTML = meaningful tags

👉 Examples:

header

nav

article

section

footer

12. Flexbox vs Grid

Flexbox

1D layout (row ya column)

Alignment easy

Grid

2D layout (rows + columns)

Complex layouts ke liye best

13. Pseudo-class in CSS

Pseudo-class = element ki special state ko style karna

👉 Examples:

:hover

:focus

:first-child

⚙️ Other Important Concepts

14. HTTP Status Codes

1xx → Informational

2xx → Success

3xx → Redirection

4xx → Client error

5xx → Server error

👉 Examples:

200 → OK

400 → Bad Request

401 → Unauthorized

403 → Forbidden

404 → Not Found

500 → Server Error

15. Basic Git Commands

git init

git add .

git commit -m "message"

git push

git pull

git fetch

Latest Stable Versions (2026)

⚛️ React JS

Current Stable Version: React 19.2

npm install react react-dom

🟢 Node.js

Current LTS Version: Node.js 22.x

Current Latest Version: Node.js 24.x

Check version:

node -v

🎨 Bootstrap

Current Stable Version: Bootstrap 5.3.x

Install:

npm install bootstrap

CDN:

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.7/dist/css/bootstrap.min.css" rel="stylesheet">

🌬️ Tailwind CSS

Current Stable Version: Tailwind CSS 4.x

Install:

npm install tailwindcss @tailwindcss/vite

🟨 JavaScript

JavaScript ka fixed version number nahi hota.

Current standard:

ECMAScript 2025 (ES2025)

Common:

ES6

ES7

ES2020+

ES2025

🌐 HTML

Current Standard: HTML5

🎨 CSS

Current Standard: CSS3

🍃 MongoDB

Current Stable Version: MongoDB 8.x

Check version:

mongod --version

Install mongoose:

npm install mongoose

🔥 Extra Important Versions (Full Stack)

⚡ Express JS

Latest Stable: Express 5.x

npm install express

🔷 TypeScript

Latest Stable: TypeScript 5.9

npm install typescript --save-dev

🅰️ Angular

Latest Stable: Angular 21

Check:

ng version

⚡ RxJS

Latest Stable: RxJS 7.8.x

npm install rxjs

⚡ Vite

Latest Stable: Vite 7.x

npm create vite@latest

MERN Stack Installation Process (Step by Step)

🧰 Step 1: Install Node.js

Install:

Node.js LTS (22.x)

👉 Website:

Node.js Official Website

Check installation:

node -v

npm -v

🍃 Step 2: Install MongoDB

👉 Download MongoDB Community Server:

MongoDB Official Website

Check:

mongod --version

⚡ Step 3: Install VS Code

👉 Download:

VS Code Official Website

⚛️ Step 4: Create MERN Project Folder

mkdir mern-project

cd mern-project

🖥️ Step 5: Create Backend (Node + Express)

🔹 Create backend folder

mkdir backend

cd backend

🔹 Initialize Node Project

npm init -y

🔹 Install Backend Packages

npm install express mongoose cors dotenv nodemon

🔹 Create Files

touch server.js

touch .env

Windows:

type nul > server.js

type nul > .env

🔹 Add Start Scripts (package.json)

"scripts": {

```
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

🔹 Basic Express Server (server.js)

```
const express = require("express");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.send("Backend Running...");
});

const PORT = 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

🔹 Run Backend

```
npm run dev
```

👉 Open:

http://localhost:5000

⚛️ Step 6: Create Frontend (React)

Go back:

cd ..

🔹 Create React App Using Vite (Recommended 🔥)

npm create vite@latest frontend

Select:

React

JavaScript

🔹 Go to Frontend

cd frontend

🔹 Install Dependencies

npm install

🔹 Start React App

npm run dev

👉 Open:

http://localhost:5173

🎨 Step 7: Install Tailwind CSS

Inside frontend:

npm install tailwindcss @tailwindcss/vite

🔹 Configure Vite

Edit vite.config.js

import { defineConfig } from 'vite'

import react from '@vitejs/plugin-react'

import tailwindcss from '@tailwindcss/vite'

```
export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

🔹 Add Tailwind in CSS

src/index.css

```
@import "tailwindcss";
```

🍃 Step 8: Connect MongoDB

🔹 .env

```
MONGO_URI=mongodb://127.0.0.1:27017/mernDB
```

🔹 Connect Database

```
const mongoose = require("mongoose");

mongoose.connect(process.env.MONGO_URI)
.then(() => console.log("MongoDB Connected"))
.catch((err) => console.log(err));
```

🔥 Final Folder Structure

```
mern-project/
|
 ├── backend/
|    ├── server.js
|    ├── package.json
|    └── .env
|
└── frontend/
     ├── src/
     ├── public/
     └── package.json
```

🚀 Run Full MERN Stack

Backend

cd backend

npm run dev

Frontend

cd frontend

npm run dev

What are the Features of React?

React ek JavaScript library hai jo UI (User Interface) banane ke liye use hoti hai.

React fast, reusable aur scalable frontend applications banane me help karta hai.

🚀 Main Features of React

1. Component-Based Architecture

React me UI ko small reusable components me divide karte hai.

👉 Example:

Navbar

Footer

Button

Card

```
function Button() {
  return <button>Click Me</button>;
}
```

✅ Reusable

✅ Easy maintenance

2. Virtual DOM

React Real DOM ki jagah Virtual DOM use karta hai.

👉 Process:

Virtual copy create hoti hai

Changes compare hote hai

Sirf changed part update hota hai

✅ Fast performance

✅ Efficient rendering

3. JSX (JavaScript XML)

JSX se JavaScript ke andar HTML-like syntax likh sakte hai.

const element = <h1>Hello React</h1>;

✅ Readable code

✅ Easy UI creation

4. One-Way Data Binding

Data parent → child direction me flow karta hai.

<Child name="Krunal" />

✅ Better control

✅ Easy debugging

5. Reusable Components

Ek component ko multiple jagah use kar sakte hai.

<Card />

<Card />

<Card />

✅ Less code

✅ Faster development

6. React Hooks

Hooks functional components me state aur lifecycle features use karne dete hai.

👉 Important Hooks:

useState

useEffect

useContext

useRef

const [count, setCount] = useState(0);

7. Fast Rendering

React sirf updated part render karta hai.

✅ Better speed

✅ Smooth UI updates

8. Declarative UI

UI ka desired state define karte hai, React automatically update karta hai.

{isLogin ? <Home /> : <Login />}

✅ Clean code

✅ Easy understanding

9. Strong Community Support

React ko Meta maintain karta hai.

✅ Huge ecosystem

✅ Large community

✅ Many libraries available

10. SEO Friendly (with Next.js)

React + Next.js use karke SSR (Server Side Rendering) possible hai.

✅ Better SEO

✅ Faster loading

React me API Kaise Fetch Karte Hai?

API fetch karne ke liye generally:

fetch()

axios

use hota hai.

🚀 Method 1: Using fetch()

✅ Example

```
import React, { useEffect, useState } from "react";

function App() {

 const [users, setUsers] = useState([]);

 useEffect(() => {

  fetch("https://jsonplaceholder.typicode.com/users")
   .then((response) => response.json())
   .then((data) => {
    setUsers(data);
   })
   .catch((error) => {
    console.log(error);
   });

 }, []);

 return (
  <div>
   <h1>User List</h1>

   {users.map((user) => (
    <p key={user.id}>{user.name}</p>
   ))}
  </div>
 );
```

}

export default App;

🔥 Flow Explain

1. useState()

Data store karne ke liye

const [users, setUsers] = useState([]);

2. useEffect()

Component load hone par API call

useEffect(() => {

}, []);

👉 [] = sirf ek baar chalega

3. fetch()

fetch("API_URL")

API request bhejta hai

4. response.json()

response.json()

JSON data convert karta hai

5. setUsers(data)

setUsers(data)

State update karta hai

📩 Output

User List

Leanne Graham

Ervin Howell

Clementine Bauch

...

🚀 Method 2: Using Axios (Most Used 🔥)

🔹 Install Axios

```
npm install axios
```

🔹 Example

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {

 const [users, setUsers] = useState([]);

 useEffect(() => {

  axios.get("https://jsonplaceholder.typicode.com/users")
   .then((response) => {
    setUsers(response.data);
   })
   .catch((error) => {
    console.log(error);
   });
```

```
  }, []);

  return (
    <div>
      <h1>Users</h1>

      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default App;
```

⚡ fetch vs axios

fetch	axios
Built-in	External library
Manual JSON conversion	Auto JSON
More code	Cleaner code
Basic	Advanced features

🎯 Interview Answer

“API fetch karne ke liye React me generally useEffect hook use karte hai taaki component load hone par API call ho. Data ko state me store karne ke liye useState use hota hai. API fetching fetch() ya axios se kar sakte hai.”

🔥 Important Interview Concepts

GET Request

Data fetch

axios.get()

POST Request

Data send

axios.post()

PUT Request

Data update

axios.put()

DELETE Request

Data delete

axios.delete()

What is Fragment in React?

React Fragment ka use multiple elements return karne ke liye hota hai without extra div.

👉 Extra unnecessary HTML avoid karta hai.

🚀 Without Fragment

```
function App() {
 return (
  <div>
   <h1>Hello</h1>
   <p>Welcome</p>
  </div>
```

```
  );
}
```

👉 Yaha extra add ho raha hai.

✅ With Fragment

```
function App() {
  return (
    <>
      <h1>Hello</h1>
      <p>Welcome</p>
    </>
  );
}
```

👉 <> </> = Fragment shortcut

🔥 Another Syntax

```
import React from "react";

function App() {
  return (
    <React.Fragment>
      <h1>Hello</h1>
      <p>Welcome</p>
    </React.Fragment>
  );
}
```

🎯 Why Use Fragment?

✅ Extra div avoid karta hai

✅ Clean HTML structure

✅ Better performance

✅ DOM lightweight hota hai

📤 Output

Hello

Welcome

Hello, my name is Krunal Chauhan. I have completed my B.E. in Computer Engineering and I have completed internship training in MERN Stack Development. During my learning and internship journey, I gained hands-on experience with MongoDB, Express.js, React.js, and Node.js. I have worked on multiple MERN stack projects, including responsive web applications and e-commerce websites, where I handled both frontend and backend development.

Along with MERN Stack development, I also have knowledge of WordPress development. I have experience creating WordPress websites, working with themes, plugins, responsive design, and website customization. I have built WordPress projects and enjoy creating user-friendly and visually appealing websites.

I also have experience with HTML, CSS, JavaScript, REST APIs, Git, Firebase, and modern development tools. I like learning new technologies and improving my technical skills through practical projects.

I consider myself a quick learner, hardworking, and adaptable person. I enjoy solving problems and working on real-world projects. Currently, I am looking for an opportunity where I can contribute my skills, continue learning, and grow professionally as a developer.